National University of Computer and Emerging Sciences
Karachi Campus

Compiler Construction (CS-4031)

Sessional-I Exam

Date: 9/22/2025

Total Time: 1hr
Total Marks: 30
Total Questions: 04

Course Instructors

Shaharbano Asif, Syed Faisal Ali, Syed Zain Ul Hassan, Anaum Hamid and Zulfiqar Ali

_____ Roll No _____ Section _____ Student Signature

Do not write below this line.

Attempt all questions.

CLO # 1: Understand the basic techniques used in compiler construction such as lexical analysis, top-down, bottom-up parsing, context-sensitive analysis, and intermediate code generation.

Question 1: Provide brief answers to the following: **(2 x 5 = 10 points)**

a) List down all the phases of compilation.
b) What is the difference between a lexeme and token?
c) What is a Symbol Table and why is it used?
d) Why is left-factoring important in parsing? Explain using an example.
e) What is the output of a Lexical Analyzer?

CLO # 2: Understand the basic data structures used in compiler construction such as abstract syntax trees, symbol tables, three-address code, and stack machines.

Question 2: Provide a single NFA for scanning the keywords **int, float, double** and **all identifiers that end with the character 'z'**. **(6 points)**

CLO # 1: Understand the basic techniques used in compiler construction such as lexical analysis, top-down, bottom-up parsing, context-sensitive analysis, and intermediate code generation.

Question 3: Consider the following grammar: **(6 points)**

```
<S> →      <V> = <E>| <S>; <S> | if <B> then <S> | if <B> then <S> else <S>
<V> →      x | y | z
<E> →      <V> | 0 | 1 | 2 | 3 | 4
<B> →      <E> = = <E>
```

Determine if this grammar is ambiguous with the help of input string: x = 1; y = 2; if x = = y then y = 3

CLO # 1: Understand the basic techniques used in compiler construction such as lexical analysis, top-down, bottom-up parsing, context-sensitive analysis, and intermediate code generation.

Question 4: Consider the following grammar and find FIRST and FOLLOW sets and nullables: **(8 points)**

```
<S> →        <two> <one>
<one> →      & <two> <one> | ε
<two> →      <three> <four>
<three> →    # <three> <four> | ε
<four> →     ( <S> ) | int
```

- Good luck -